

Байт-Субст

Байт-Когнітивний Субстрат

Зміст

1 Вступ: Проблема сучасних архітектур штучного інтелекту	3
2 Філософські та епістемологічні засади	4
2.1 Феноменологія сприйняття: від сенсорних даних до розуміння . .	4
2.2 Біологічні аналогії: нервова система та сирі сигнали	4
2.3 Принцип мінімальної достатності	5
3 Архітектурна концепція: Байт-Когнітивний Субстрат	5
3.1 Огляд архітектури	5
3.2 Байт як атом пізнання	6
3.3 Чому не біт і не токен?	6
4 Математичний формалізм	7
4.1 Байтовий простір та його структура	7
4.2 Тензори байтової взаємодії	8
4.3 Польові рівняння байтового субстрату	9
4.4 Інформаційна геометрія байтових розподілів	10
4.5 Динаміка самоорганізації	10
5 Шари конвертації: від байтів до абстракцій	11
5.1 Принципи конвертації	11
5.2 Адаптивні шари перетворення	12
5.3 Модальність-незалежна архітектура	13
6 Механізми навчання та адаптації	13
6.1 Принцип вільної енергії на байтовому рівні	13
6.2 Інформаційне вузьке місце як дизайн-принцип	14
6.3 Емерджентна оптимізація	14
7 Порівняльний аналіз з існуючими підходами	15
7.1 Трансформери	15
7.2 Згорткові нейронні мережі	15
7.3 Рекурентні архітектури	15
7.4 Нейроеволюційні підходи	16
8 Шляхи реалізації	16
8.1 Обчислювальні вимоги	16
8.2 Апаратна архітектура	16
8.3 Етапи розробки	17

9	Накопичення досвіду та пам'ять	17
9.1	Кристалізована пам'ять (довгострокова)	17
9.2	Робоча пам'ять (короткострокова)	18
9.3	Впізнавання через атракторні басейни	18
10	Контекстна архітектура	19
10.1	Локальний контекст: польова взаємодія	19
10.2	Послідовний контекст: контекстний резонанс	19
10.3	Накопичувальний контекст: гладкий інерційний стан	20
11	Часові масштаби та їх узгодження	20
11.1	Елементарний такт та шкала часу	20
11.2	Умова розділення масштабів	20
11.3	Адаптивний елементарний такт	21
12	Перенесення знань між модальностями	21
12.1	Механізм 1: Структурний ізоморфізм	21
12.2	Механізм 2: Універсальний байтовий скаффолд	22
12.3	Механізм 3: Ланцюгове перенесення	22
12.4	Механізм 4: Мовне заземлення	22
13	Зростання складності абстракцій: автокаталітичне розщеплення	23
13.1	Крок 0: Детекція потреби	23
13.2	Крок 1: Побудова графа подібності	23
13.3	Крок 2: Спектральна кластеризація	23
13.4	Крок 3: Створення нового рівня	23
13.5	Крок 4: Калібрація	24
13.6	Крок 5: Валідація через $\Delta\mathcal{F}_{\text{free}}$	24
13.7	Зрощення надмірних рівнів	24
14	Потенційні проблеми та шляхи їх вирішення	24
14.1	Проблема масштабованості байтового поля	24
14.2	Проблема вибору початкових параметрів	25
14.3	Проблема "дитячої амнезії"	25
14.4	Проблема катастрофічного забування	25
14.5	Проблема шуму та нерелевантних даних	25
14.6	Проблема упередженості кодування	26
14.7	Проблема обчислювальної ефективності атракторних басейнів	26
15	Висновки та перспективи	26

1 Вступ: Проблема сучасних архітектур штучного інтелекту

Сучасні архітектури штучного інтелекту (ШІ) побудовані на фундаментальній передумові, яка рідко піддається сумніву: необхідності попередньої абстракції вхідних даних. Трансформери оперують токенами — дискретними символами, отриманими в результаті роботи токенизатора; згорткові мережі працюють із попередньо структурованими тензорами фіксованої розмірності; рекурентні моделі поглинають послідовності елементів, які вже пройшли стадію символного або числового кодування. У кожному з цих випадків система ніколи не зустрічається з сирий інформацією — вона завжди отримує вже оброблений, відфільтрований та інтерпретований набір абстракцій.

Ця ситуація створює глибокий парадокс. Чим більше ми абстрагуємо вхідні дані, тим більше ми втрачаємо саме ту інформацію, яка могла б стати основою для виникнення справжнього інтелекту. Біологічні організми не отримують попередньо токенизованих повідомлень від навколишнього середовища — їхні сенсорні системи сприймають безперервні потоки сирих сигналів: фотони, тиску повітря на барабанну перетинку, хімічні градієнти в рецепторних клітинах. І саме з цього сирого, неструктурованого матеріалу виникає складна когнітивна поведінка.

Центральна теза даної роботи формулюється так: **якщо ми прагнемо створити систему, здатну до справжнього інтелекту, ми не можемо давати їй спеціально підготовлені абстракції, відфільтровані від реальності.** Замість цього система повинна отримати доступ до повноти інформації, причому у її найменш структурованій формі — на рівні байтів. Байт є універсальною одиницею цифрового представлення: будь-які дані — текст, зображення, звук, відео, програмний код — зводяться до послідовностей байтів. Ця універсальність робить байт ідеальним субстратом для когнітивної архітектури нового типу.

У даній роботі ми пропонуємо архітектуру **Байт-Когнітивний Субстрат** (БКС), яка базується на трьох ключових принципах. Перший — принцип повноти субстрату: система оперує виключно байтовими послідовностями, без будь-якої попередньої обробки або токенизації. Другий — принцип польової взаємодії: байти взаємодіють між собою через динамічні поля, аналогічні фізичним полям, що дозволяє виникати емерджентним структурам. Третій — принцип адаптивної конвертації: система сама формує шари абстракцій зверху вниз від байтового субстрату, а не отримує їх ззовні.

Робота має таку структуру. У розділі 2 ми розглядаємо філософські та епістемологічні засади запропонованого підходу. Розділ 3 містить детальний опис архітектурної концепції БКС. У розділі 4 розробляється строгий математичний формалізм, включаючи тензорний аналіз байтової взаємодії, польові рівняння та інформаційну геометрію. Розділ 5 описує механізми конвертації від байтів до вищих абстракцій. Розділ 6 присвячений механізмам навчання та адаптації. Розділ 7 містить порівняльний аналіз з існуючими підходами, а розділ 8 — шляхи реалізації.

2 Філософські та епістемологічні засади

2.1 Феноменологія сприйняття: від сенсорних даних до розуміння

Феноменологічна традиція, починаючи від Гуссерля та закінчуючи Мерло-Понті, наголошує на фундаментальній ролі досвіду першого порядку у формуванні пізнання. Сприйняття не є пасивним прийомом готових значень — це активний процес конституювання смислу з сирого сенсорного матеріалу. Мерло-Понті у своїй праці “Феноменологія сприйняття” показує, що сприйняття форми, кольору, простору є результатом складної взаємодії між сенсорним полем та тілесною схемою суб’єкта. Немає “токенізатора”, який попередньо перетворює фотони на “червоний квадрат” — це перетворення відбувається всередині самої системи сприйняття.

Аналогічно, в контексті III ми стверджуємо, що неможливо побудувати систему, яка дійсно “розуміє” інформацію, якщо ця інформація надходить до неї вже у попередньо структурованій, абстрагованій формі. Токенізація — це не просто технічна операція стиснення даних; це акт інтерпретації, яка накладає на дані структуру, визначену людиною-розробником. Коли токенізатор розбиває слово “невідомість” на підслова “не” + “відомість”, він вже здійснює лінгвістичний аналіз, результат якого система отримує як дані, а не як власне відкриття. Це фундаментально обмежує здатність системи до самостійного формування абстракцій.

2.2 Біологічні аналогії: нервова система та сирий сигнали

Біологічна нервова система є переконливим прикладом архітектури, яка працює з сирими, неструктурованими даними. Фоторецептори сітківки перетворюють окремі фотони на електричні потенціали; волоскові клітини внутрішнього вуха перетворюють механічні коливання на нервові імпульси; хеморецептори мовних сосочків генерують сигнали у відповідь на молекулярні взаємодії. У жодному з цих випадків сенсорний орган не виконує “токенізації” — він передає сирий сигнал, а подальша обробка відбувається в послідовних шарах нервової системи.

Важливо підкреслити, що біологічні сенсорні системи не просто “пропускають” сирий сигнал — вони здійснюють мінімальне перетворення, яке зберігає максимальну кількість інформації про стимул. Фоторецептор не вирішує, чи є стимул “червоним” чи “синім” — він генерує градуальний потенціал, пропорційний інтенсивності світла в певному спектральному діапазоні. Інтерпретація кольору виникає емерджентно в результаті взаємодії між тисячами таких сигналів у зоровій корі.

Ця біологічна реальність безпосередньо підтримує наш підхід: байтовий субстрат є цифровим аналогом сирого сенсорного сигналу. Подібно до того, як біологічна система відкриває структуру в сенсорному потоці, БКС має відкривати структуру в байтовому потоці.

2.3 Принцип мінімальної достатності

Ми формулюємо принцип мінімальної достатності: система повинна отримувати мінімально достатний рівень абстракції вхідних даних, де “мінімально достатний” означає найнижчий рівень, на якому ще зберігається повнота інформації. Для цифрових систем цей рівень є рівнем байтів. Байт — це найменша адресована одиниця даних, яка містить 8 бітів і може приймати 256 значень. Будь-яка інформація, що може бути представлена в цифровій формі, зводиться до послідовності байтів.

Принцип мінімальної достатності не означає, що система повинна обробляти байти “як є” без жодної внутрішньої трансформації. Навпаки, система повинна мати можливість самостійно формувати абстракції з байтового субстрату. Ключова відмінність полягає в тому, хто здійснює абстрагування: у традиційних підходах це робить розробник через токенизацію та попередню обробку; у нашому підході це робить сама система через механізми самоорганізації.

Цей принцип має глибокий зв’язок з теорією інформаційного вузького місця (Information Bottleneck), яка стверджує, що оптимальне представлення даних зберігає максимальну інформацію про цільову змінну при мінімальній складності самого представлення. У нашому випадку “цільова змінна” — це інтелектуальна поведінка, а “складність представлення” мінімізується тим, що система починає з найпростішого можливого представлення — байтів.

3 Архітектурна концепція: Байт-Когнітивний Субстрат

3.1 Огляд архітектури

Архітектура Байт-Когнітивного Субстрату (БКС) складається з чотирьох основних компонентів, кожен з яких виконує визначену функцію в загальному процесі перетворення байтових потоків на інтелектуальну поведінку (рис. 1).

Перший компонент — Байтовий Субстрат ($\mathcal{B}$) — є фундаментом архітектури. Він являє собою простір байтових послідовностей, які поступають у систему без будь-якої попередньої обробки. Кожен байт приймає значення з множини $\{0, 1, \dots, 255\}$, а послідовність байтів довжиною N є елементом простору $\mathcal{B}^N$. Байтовий субстрат не здійснює жодної інтерпретації даних — він є чистим носієм інформації.

Другий компонент — Польова система взаємодії ($\mathcal{F}$) — визначає, як байти впливають один на одного. Кожна пара байтів у послідовності породжує поле взаємодії, інтенсивність якого залежить від відстані між байтами, їхніх значень та контексту оточення. Формально, це описується тензорами взаємодії, які ми детально розглянемо в розділі 4.

Третій компонент — Механізм самоорганізації — забезпечує виникнення структурованих патернів з хаотичної байтової взаємодії. Цей механізм базується на принципі мінімізації вільної енергії, адаптованому для байтового рівня. Коли інтенсивність взаємодії між певними групами байтів перевищує порогове значення, утворюються когнітивні кластери — стабільні конфігурації, які відповідають виявленим шаблонам даних.

Четвертий компонент — Шари адаптивної конвертації — перетворюють когнітивні кластери на вищорівневі представлення. На відміну від фіксованих

шарів нейронних мереж, конвертаційні шари БКС формуються динамічно: система сама визначає, які кластери об'єднувати, які абстракції формувати, і які модальності розрізняти. Конвертація відбувається за принципом зворотного зв'язку: вищі шари сигналізують нижчим про релевантність тих чи інших патернів, що призводить до перерозподілу уваги на рівні субстрату.

Рівень	Компонент	Функція
4	Шари адаптивної конвертації	Формування вищих абстракцій
3	Механізм самоорганізації	Утворення когнітивних кластерів
2	Польова система взаємодії	Взаємний вплив байтів
1	Байтовий субстрат	Зберігання та подача сирих даних

Рис. 1: Ієрархія компонентів архітектури БКС.

3.2 Байт як атом пізнання

Вибір байта як фундаментальної одиниці обробки не є довільним. Байт володіє низкою унікальних властивостей, які роблять його ідеальним кандидатом на роль “атома пізнання” в архітектурі III.

По-перше, байт є універсальним. Будь-яка інформація, що може бути оцифрована, представлена у вигляді послідовності байтів. Текст у кодуванні UTF-8 — це послідовність байтів; зображення у форматі PNG — це послідовність байтів; аудіо у форматі WAV — це послідовність байтів; навіть виконуваний код — це послідовність байтів. Ця універсальність означає, що система, здатна обробляти байти, потенційно здатна обробляти будь-які дані без модифікації архітектури.

По-друге, байт є мінімально структурованим. Він містить рівно стільки інформації, скільки необхідно для адресації — 8 бітів, 256 можливих значень. Це найменша одиниця, яка ще зберігає змістовну інформацію (на відміну від біта, який бінарний і тому занадто бідний для самостійної обробки). Байт достатньо виразний, щоб кодувати широкий спектр значень, але недостатньо виразний, щоб нав'язувати якусь конкретну інтерпретацію.

По-третє, байтове представлення є модально-незалежним. Не існує “текстового байта” чи “зображувального байта” — байт є байтом незалежно від того, яку інформацію він кодує. Це дозволяє системі БКС обробляти різні модальності єдиним механізмом, а відмінності між модальностями виявляються емерджентно через різні патерни байтових послідовностей.

3.3 Чому не біт і не токен?

Важливо розрізняти байт від двох інших потенційних кандидатів на роль фундаментальної одиниці: біта та токена.

Біт занадто примітивний. Ізольований біт несе лише 1 біт інформації (у значенні Шеннона) і має лише 2 можливих стани. Обробка даних на бітовому рівні вимагала б роботи з послідовностями довжиною у 8 разів більшими, ніж при байтовій обробці, без жодної додаткової вигоди. Біт є аналогом окремого нейронного імпульсу (спайк) у біологічній системі — це занадто дрібномасштабна

одиниця для формування когнітивних структур. Біологічні системи не оперують окремими спайками як одиницями пізнання; вони працюють з патернами спайкової активності, які є аналогами байтових послідовностей.

Токен, навпаки, занадто абстрактний. Токен вже є результатом інтерпретації даних людиною або алгоритмом. Токен “cat” в англійській мові кодує конкретне лінгвістичне поняття; токен “[IMG_PATCH_3]” у візуальній моделі кодує певну область зображення. У кожному випадку токенізація втрачає інформацію про альтернативні способи структурування даних. Крім того, токенізація є модально-залежною: токенізатор для тексту принципово відрізняється від токенізатора для зображень, що унеможливорює єдину архітектуру.

Байт займає “золоту середину” між цими двома крайнощами: він достатньо виразний для кодування змістовної інформації, але недостатньо структурований для нав’язування інтерпретації.

4 Математичний формалізм

У цьому розділі ми розробляємо строгий математичний апарат для опису архітектури БКС. Ми починаємо з визначення байтового простору та його структури, потім вводимо тензори взаємодії, формулюємо польові рівняння, і завершуємо інформаційною геометрією та динамікою самоорганізації.

4.1 Байтовий простір та його структура

Визначення 4.1 (Байтовий алфавіт). Байтовий алфавіт $\mathbb{B}$ є скінченною множиною $\{0, 1, \dots, 255\}$, елементи якої називаються байтовими значеннями.

Визначення 4.2 (Байтовий потік). Байтовий потік довжиною $N \in \mathbb{N}$ є послідовність $s = (b_1, b_2, \dots, b_N)$, де $b_i \in \mathbb{B}$ для всіх $i = 1, \dots, N$. Множина всіх байтових потоків довжини N позначається $\mathbb{B}^N$.

Визначення 4.3 (Байтовий простір). Байтовий простір $\mathcal{B}$ є нескінченновимірним простором, утвореним об’єднанням $\bigcup_{N=1}^{\infty} \mathbb{B}^N$. Елементи $\mathcal{B}$ є байтовими потоками довільної довжини.

Для кожної позиції i у байтовому потоці s ми визначаємо одногарячне кодування байтового значення b_i як вектор $\mathbf{e}_i \in \{0, 1\}^{256}$, де $(\mathbf{e}_i)_k = \delta_{k, b_i}$. Це дозволяє представити будь-який байтовий потік як послідовність векторів у $\mathbb{R}^{256}$:

$$s \mapsto (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N), \quad \mathbf{e}_i \in \mathbb{R}^{256}. \quad (1)$$

Однак одногарячне кодування є лише початковим представленням. У ході обробки система формує ембединг-представлення кожного байта — вектор у просторі більш високої розмірності $d \gg 256$, який враховує контекст:

$$\mathbf{h}_i = \phi(b_i, \mathbf{C}_i) \in \mathbb{R}^d, \quad (2)$$

де $\phi : \mathbb{B} \times \mathbb{R}^{256 \times d} \rightarrow \mathbb{R}^d$ — функція ембедингу, а $\mathbf{C}_i$ — контекстна матриця, яка залежить от оточуючих байтів. На відміну від трансформерів, де ембединг є статичним (або обумовленим лише позицією), у БКС ембединг є динамічним і залежить від стану всієї польової системи.

4.2 Тензори байтової взаємодії

Ключовим елементом архітектури БКС є тензор взаємодії, який визначає силу та характер впливу одного байта на інший. Цей тензор є аналогом матриці уваги (attention matrix) у трансформерах, але має принципово іншу структуру та динаміку.

Визначення 4.4 (Тензор байтової взаємодії). Для байтового потоку $s = (b_1, \dots, b_N)$ тензор байтової взаємодії $\mathcal{T} \in \mathbb{R}^{N \times N \times 256 \times 256}$ визначається як:

$$\mathcal{T}_{ijkm} = f(b_i, b_j, |i - j|, \mathbf{h}_i, \mathbf{h}_j) \cdot \sigma(k - b_i) \cdot \sigma(m - b_j), \quad (3)$$

де $f: \mathbb{B} \times \mathbb{B} \times \mathbb{N} \times \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ — функція інтенсивності взаємодії, а $\sigma: \mathbb{Z} \rightarrow \{0, 1\}$ — одинична функція (індикатор).

Символіка в рівнянні (3) має такий зміст. Індекси i, j визначають позиції байтів у потоці; індекси k, m визначають конкретні байтові значення. Таким чином, $\mathcal{T}_{ijkm}$ кодує силу взаємодії між i -м байтом (якщо він має значення k) та j -м байтом (якщо він має значення m). Функція σ відіграє роль селектора: ненульовий внесок дають лише ті компоненти тензора, які відповідають фактичним значенням байтів.

Функція інтенсивності взаємодії f має наступну структуру:

$$f(b_i, b_j, r, \mathbf{h}_i, \mathbf{h}_j) = \underbrace{\alpha(r)}_{\text{відстань}} \cdot \underbrace{\beta(b_i, b_j)}_{\text{сумісність значень}} \cdot \underbrace{\gamma(\mathbf{h}_i, \mathbf{h}_j)}_{\text{контекстна спорідненість}}, \quad (4)$$

де:

- $\alpha: \mathbb{N} \rightarrow \mathbb{R}_+$ — функція затухання відстані. На відміну від механізму уваги трансформерів, де відстань впливає лише через позиційне кодування, у БКС ми використовуємо експоненційне затухання з адаптивним параметром:

$$\alpha(r) = \exp\left(-\frac{r}{\lambda(\mathbf{h}_i, \mathbf{h}_j)}\right), \quad (5)$$

де $\lambda(\mathbf{h}_i, \mathbf{h}_j) > 0$ — довжина взаємодії, яка залежить від контексту. Це дозволяє системі динамічно регулювати “горизонт” взаємодії: для високоструктурованих ділянок даних λ може бути малою (локальна взаємодія), а для монотонних або періодичних ділянок — великою (глобальна взаємодія).

- $\beta: \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{R}_+$ — функція сумісності байтових значень. Вона кодує “афінитет” між різними байтовими значеннями. Наприклад, значення $0x20$ (пробіл у ASCII) та $0x41$ (“A”) можуть мати високу сумісність у текстовому контексті, але низьку — у бінарному. Функція β обчислюється як:

$$\beta(b_i, b_j) = \mathbf{u}_{b_i}^\top \mathbf{W}_\beta \mathbf{u}_{b_j}, \quad (6)$$

де $\mathbf{u}_k \in \mathbb{R}^{d_\beta}$ — вивчені вектори байтових значень, а $\mathbf{W}_\beta \in \mathbb{R}^{d_\beta \times d_\beta}$ — матриця сумісності, яка також є параметром, що навчається.

- $\gamma: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}_+$ — функція контекстної спорідненості, аналогічна скалярному добутку в механізмі уваги:

$$\gamma(\mathbf{h}_i, \mathbf{h}_j) = \frac{\exp(\mathbf{h}_i^\top \mathbf{A} \mathbf{h}_j / \sqrt{d})}{\sum_{k=1}^N \exp(\mathbf{h}_i^\top \mathbf{A} \mathbf{h}_k / \sqrt{d})}, \quad (7)$$

де $\mathbf{A} \in \mathbb{R}^{d \times d}$ — матриця контекстної уваги.

4.3 Польові рівняння байтового субстрату

Байтовий субстрат описується польовим станом, який еволюціонує в часі під впливом взаємодій. Ми визначаємо поле байтової активації $\Phi : \{1, \dots, N\} \times \mathbb{B} \times \mathbb{R}_+ \rightarrow \mathbb{R}$ як функцію позиції, байтового значення та часу.

Визначення 4.5 (Поле байтової активації). Поле байтової активації $\Phi(i, k, t)$ визначає “рівень збудження” байтового значення k на позиції i у момент часу t . Початковий стан поля задається:

$$\Phi(i, k, 0) = \begin{cases} 1, & \text{якщо } b_i = k, \\ 0, & \text{інакше.} \end{cases} \quad (8)$$

Еволюція поля описується польовим рівнянням БКС, яке є дискретним аналогом рівняння реакції-дифузії:

$$\frac{\partial \Phi(i, k, t)}{\partial t} = \underbrace{D_k \sum_{j \in \mathcal{N}(i)} [\Phi(j, k, t) - \Phi(i, k, t)]}_{\text{дифузія}} + \underbrace{R_k(\Phi(i, \cdot, t), \theta_i)}_{\text{реакція}} - \underbrace{\mu \Phi(i, k, t)}_{\text{затухання}}, \quad (9)$$

де:

- $D_k > 0$ — коефіцієнт дифузії для байтового значення k , який визначає швидкість розповсюдження активації. Різні байтові значення можуть мати різні коефіцієнти дифузії, що відображає їхню різну “мобільність” у байтовому просторі (наприклад, значення-розділювачі можуть дифундувати швидше, ніж значення-дані).
- $\mathcal{N}(i)$ — околиця позиції i , яка може бути як простою ($\mathcal{N}(i) = \{i - 1, i + 1\}$ — найближчі сусіди), так і адаптивною, залежною від стану поля.
- $R_k : \mathbb{R}^{256} \times \Theta \rightarrow \mathbb{R}$ — функція реакції, яка визначає нелінійну взаємодію між різними байтовими значеннями на тій самій позиції. Вона залежить від вектора активації $\Phi(i, \cdot, t) \in \mathbb{R}^{256}$ та локальних параметрів θ_i .
- $\mu > 0$ — коефіцієнт затухання, який забезпечує стабільність польової динаміки та запобігає нескінченному зростанню активації.

Функція реакції R_k є ключовим елементом, що забезпечує виникнення структурованих патернів. Ми визначаємо її через локальну функцію енергії:

$$R_k(\phi, \theta) = -\frac{\partial \mathcal{E}_k(\phi, \theta)}{\partial \phi_k}, \quad (10)$$

де $\mathcal{E}_k : \mathbb{R}^{256} \times \Theta \rightarrow \mathbb{R}$ — локальна енергія для байтового значення k . Конкретний вибір $\mathcal{E}_k$ визначає типи патернів, які можуть виникати в системі. Для формування когнітивних кластерів ми використовуємо подвійну потенціальну яму (double-well potential), аналогічну рівнянню Аллена-Кана:

$$\mathcal{E}_k(\phi_k, \theta_k) = a_k (\phi_k^2 - \theta_k)^2, \quad (11)$$

де $a_k > 0$ — параметр глибини потенціальної ями, а $\theta_k > 0$ — параметр, що визначає стабільні стани (рівноважні значення активації). При $\theta_k > 0$ система має два стабільні стани: $\phi_k = \pm \sqrt{\theta_k}$, що відповідає “увімкненому” та “вимкненому” станам активації байтового значення k . Перехід між цими станами під дією взаємодії з сусідніми байтами є механізмом формування кластерів.

4.4 Інформаційна геометрія байтових розподілів

Для опису статистичної структури байтового субстрату ми використовуємо апарат інформаційної геометрії. Розглянемо сімейство ймовірнісних розподілів над байтовими значеннями:

$$\mathcal{P} = \{p_\theta : \theta \in \Theta \subset \mathbb{R}^m\}, \quad p_\theta(k) = \frac{\exp(\theta^\top \mathbf{f}(k))}{Z(\theta)}, \quad k \in \mathbb{B}, \quad (12)$$

де $\mathbf{f} : \mathbb{B} \rightarrow \mathbb{R}^m$ — вектор достатніх статистик, а $Z(\theta) = \sum_{k=0}^{255} \exp(\theta^\top \mathbf{f}(k))$ — статистична сума.

Визначення 4.6 (Метрика Фішера для байтового простору). Метрика Фішера на многовиді $\mathcal{P}$ визначається як:

$$g_{ij}(\theta) = \mathbb{E}_{p_\theta} \left[\frac{\partial \log p_\theta(k)}{\partial \theta_i} \cdot \frac{\partial \log p_\theta(k)}{\partial \theta_j} \right] = \sum_{k=0}^{255} p_\theta(k) \frac{\partial \log p_\theta(k)}{\partial \theta_i} \frac{\partial \log p_\theta(k)}{\partial \theta_j}. \quad (13)$$

Метрика Фішера визначає природну геометрію простору байтових розподілів: відстань між двома розподілами p_θ та $p_{\theta'}$ вимірюється через дивергенцію Кульбака-Лейблера:

$$D_{\text{KL}}(p_\theta \| p_{\theta'}) = \sum_{k=0}^{255} p_\theta(k) \log \frac{p_\theta(k)}{p_{\theta'}(k)}. \quad (14)$$

Зв'язок між польовою динамікою та інформаційною геометрією встановлюється через принцип мінімізації дії. Визначимо функціонал дії для байтового субстрату:

$$S[\Phi] = \int_{t_0}^{t_1} \left[\sum_{i,k} \frac{1}{2} \left(\frac{\partial \Phi(i,k,t)}{\partial t} \right)^2 - \mathcal{E}_{\text{total}}(\Phi, t) \right] dt, \quad (15)$$

де $\mathcal{E}_{\text{total}}$ — повна енергія системи, що включає локальні енергії (11) та енергію взаємодії, що визначається тензором $\mathcal{T}$.

Теорема 4.1 (Принцип найменшої дії). Рівняння еволюції поля (9) є рівнянням Ейлера-Лагранжа для функціоналу дії (15), тобто:

$$\frac{\delta S}{\delta \Phi(i,k,t)} = 0 \implies \frac{\partial^2 \Phi(i,k,t)}{\partial t^2} = - \frac{\partial \mathcal{E}_{\text{total}}}{\partial \Phi(i,k,t)}. \quad (16)$$

У Overdamped режимі (малий коефіцієнт інерції) це зводиться до (9).

4.5 Динаміка самоорганізації

Самоорганізація байтового субстрату — це процес, у ході якого з первісно хаотичного стану поля виникають структуровані конфігурації (кластери). Цей процес керується принципом мінімізації вільної енергії, адаптованим з теорії Карла Фрістона.

Визначення 4.7 (Вільна енергія байтового субстрату). Вільна енергія системи у момент часу t визначається як:

$$\mathcal{F}_{\text{free}}(t) = \underbrace{\mathcal{E}_{\text{total}}(\Phi, t)}_{\text{внутрішня енергія}} - T \underbrace{S(\Phi, t)}_{\text{ентропія}}, \quad (17)$$

де $T > 0$ — параметр температури, а $S(\Phi, t)$ — ентропія Шеннона поля активації:

$$S(\Phi, t) = - \sum_{i=1}^N \sum_{k=0}^{255} \tilde{\phi}(i, k, t) \log \tilde{\phi}(i, k, t), \quad (18)$$

причому $\tilde{\phi}(i, k, t) = \frac{|\Phi(i, k, t)|}{\sum_{m=0}^{255} |\Phi(i, m, t)|}$ — нормалізована активація.

Теорема 4.2 (Про самоорганізацію). Якщо функція реакції R_k задовольняє умову $\frac{\partial R_k}{\partial \phi_k} < 0$ у деякій області простору станів, а коефіцієнт дифузії D_k достатньо малий, то існує таке критичне значення температури $T_c > 0$, що при $T < T_c$ система зазнає фазового переходу, у результаті якого утворюються стабільні когнітивні кластери.

Наближене обґрунтування. Розглянемо лінеаризацію рівняння (9) навколо однорідного стану Φ_0 . Вносячи збурення $\delta\Phi(i, k, t) = \Phi(i, k, t) - \Phi_0(k)$ та застосовуючи перетворення Фур'є за просторовою змінною, отримуємо дисперсійне співвідношення:

$$\omega(q, k) = -D_k q^2 + \left. \frac{\partial R_k}{\partial \phi_k} \right|_{\phi_0} - \mu, \quad (19)$$

де q — хвильове число. Нестійкість (і, отже, самоорганізація) виникає, коли $\omega(q, k) > 0$ для деяких q та k . При $T \rightarrow T_c$ знизу параметр $\partial R_k / \partial \phi_k$ перевищує поріг $D_k q_{\max}^2 + \mu$, що веде до експоненційного зростання збурень з хвильовим числом $q_{\max}$ — характерним масштабом кластерів. $\square$

Визначення 4.8 (Когнітивний кластер). Когнітивний кластер $C = (\mathcal{I}, \mathbf{p})$ — це підмножина позицій $\mathcal{I} \subseteq \{1, \dots, N\}$ з асоційованим розподілом байтових значень $\mathbf{p} \in \mathbb{R}^{256}$, така що:

- (i) Для всіх $i \in \mathcal{I}$ та $j \in \mathcal{I}$ виконується: $D_{\text{KL}}(\tilde{\phi}(i, \cdot, t) \| \tilde{\phi}(j, \cdot, t)) < \epsilon$ (внутрішня когерентність).
- (ii) Для всіх $i \in \mathcal{I}$ та $j \notin \mathcal{I}$ виконується: $D_{\text{KL}}(\tilde{\phi}(i, \cdot, t) \| \tilde{\phi}(j, \cdot, t)) \geq \delta$ (зовнішня відмінність).
- (iii) Множина $\mathcal{I}$ є зв'язною (суміжною) у графі взаємодії.

Параметри $\epsilon < \delta$ визначають жорсткість кластеризації.

Процес самоорганізації можна зрозуміти як послідовність фазових переходів. На першій стадії (висока температура T) система перебуває в “дисперсному” стані — активація розподілена рівномірно. При зниженні T нижче $T_{c,1}$ виникають локальні кластери, що відповідають найпростішим патернам (повторення, межі). Далі при $T < T_{c,2}$ кластери об'єднуються у суперструктури, що відповідають складнішим шаблонам. Цей процес продовжується ієрархічно, утворюючи багаторівневу структуру абстракцій.

5 Шари конвертації: від байтів до абстракцій

5.1 Принципи конвертації

Конвертація у контексті БКС — це процес перетворення когнітивних кластерів нижчого рівня на представлення вищого рівня. На відміну від шарів нейрон-

ної мережі, де перетворення є фіксованим (визначеним архітектурою та вагами), конвертація у БКС є адаптивною: система сама визначає, які кластери об'єднувати, які ознаки виділяти, та які рівні абстракції формувати.

Ми виділяємо три фундаментальні принципи конвертації.

Принцип ієрархічної агрегації: кожен рівень конвертації об'єднує кластери попереднього рівня у нові структури, використовуючи метрику спорідненості, яка базується на дивергенції Кульбака-Лейблера між розподілами кластерів. Формально, якщо $C_1, \dots, C_M$ — кластери рівня l , то кластер рівня $l + 1$ утворюється як:

$$C^{(l+1)} = \text{Agg}(C_{i_1}^{(l)}, \dots, C_{i_m}^{(l)}; w_{i_1}, \dots, w_{i_m}), \quad (20)$$

де ваги w_{i_j} визначаються через контекстну спорідненість γ (див. (7)), а функція агрегації Agg формує новий розподіл як зважену суму:

$$\mathbf{p}^{(l+1)} = \sum_{j=1}^m w_{i_j} \mathbf{p}_{i_j}^{(l)}, \quad \sum_{j=1}^m w_{i_j} = 1. \quad (21)$$

Принцип зворотного зв'язку: конвертаційні шари вищого рівня надсилають сигнали вниз до субстрату, модифікуючи параметри польової системи (D_k, θ_k, μ) та тензорів взаємодії ($\mathbf{W}_\beta, \mathbf{A}$). Це дозволяє системі “фокусувати увагу” на тих ділянках байтового потоку, які є найбільш інформативними з точки зору поточного контексту.

Принцип модальної ідентифікації: система автоматично виявляє модальність даних (текст, зображення, аудіо, тощо) через статистичні властивості байтових розподілів, без жодного зовнішнього сигналу. Наприклад, текстові дані в UTF-8 мають характерне розподілення байтових значень (переважання значень у діапазоні 0x20–0x7E для ASCII), тоді як стиснуті зображення мають приблизно рівномірний розподіл байтів.

5.2 Адаптивні шари перетворення

Кожен конвертаційний шар ℓ визначається трійкою $\mathcal{C}_\ell = (\mathcal{K}_\ell, \mathcal{T}_\ell, \mathcal{R}_\ell)$, де:

- $\mathcal{K}_\ell$ — множина когнітивних кластерів рівня ℓ ;
- $\mathcal{T}_\ell : \mathcal{K}_\ell \rightarrow \mathbb{R}^{d_\ell}$ — функція перетворення, яка перетворює кластер на векторне представлення у просторі розмірності d_ℓ ;
- $\mathcal{R}_\ell : \mathbb{R}^{d_\ell} \times \mathbb{R}^{d_{\ell+1}} \rightarrow \mathbb{R}$ — функція релевантності, яка визначає, наскільки даний кластер рівня ℓ релевантний для кластера рівня $\ell + 1$.

Функція перетворення $\mathcal{T}_\ell$ обчислюється рекурсивно. На найнижчому рівні ($\ell = 0$) вона є просто розподілом байтових значень у кластері:

$$\mathcal{T}_0(C) = \mathbf{p}_C \in \mathbb{R}^{256}. \quad (22)$$

На вищих рівнях вона обчислюється через графову нейронну мережу на графі кластерів:

$$\mathcal{T}_\ell(C) = \sigma \left(\mathbf{W}_\ell^{(1)} \sum_{C' \in \mathcal{N}(C)} \frac{\mathcal{R}_{\ell-1}(C', C)}{\sqrt{|\mathcal{N}(C)| \cdot |\mathcal{N}(C')|}} \mathcal{T}_{\ell-1}(C') + \mathbf{b}_\ell \right), \quad (23)$$

де $\mathbf{W}_\ell^{(1)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$ та $\mathbf{b}_\ell \in \mathbb{R}^{d_\ell}$ — параметри, що навчаються, σ — функція активації, а $\mathcal{N}(C)$ — множина сусідніх кластерів у графі взаємодії.

5.3 Модальність-незалежна архітектура

Ключовою перевагою БКС є її модальність-незалежність. Система не потребує різних “голів” для різних модальностей — замість цього модальність виявляється автоматично через статистичні властивості байтових розподілів.

Для формалізації цього процесу ми визначаємо модальний ідентифікатор як функцію від розподілу байтових значень у вікні розміром W :

$$\mathcal{M}(\mathbf{p}) = \arg \max_{m \in \mathcal{M}} \mathbb{P}(m|\mathbf{p}), \quad (24)$$

де $\mathcal{M} = \{\text{текст, зображення, аудіо, бінарний, структурований, \dots}\}$ — множина модальностей, а $\mathbb{P}(m|\mathbf{p})$ оцінюється через баєсівське правило з апіорними розподілами модальностей.

Після ідентифікації модальності система адаптує параметри конвертаційних шарів: для тексту можуть формуватися лексичні та синтаксичні абстракції, для зображень — просторові та текстурні, для аудіо — тимчасові та спектральні. Важливо, що ця адаптація відбувається через навчання, а не через жорстке програмування: система вчиться розпізнавати модальності та формувати відповідні абстракції на основі власного досвіду обробки байтових потоків.

6 Механізми навчання та адаптації

6.1 Принцип вільної енергії на байтовому рівні

Навчання у БКС базується на принципі мінімізації вільної енергії, адаптованому з предиктивного кодування Фрістона. Основна ідея полягає в тому, що система постійно генерує передбачення про майбутні байтові значення на основі поточного стану поля, і навчання відбувається через мінімізацію розбіжності між передбаченням та реальністю.

Визначимо генеративну модель байтового субстрату як ієрархію латентних змінних $\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \dots, \mathbf{z}^{(L)}$, де кожен рівень генерує передбачення для рівня нижче:

$$p(\mathbf{z}^{(\ell)}|\mathbf{z}^{(\ell+1)}) = \mathcal{N}(\mathbf{z}^{(\ell)}; \mathbf{W}^{(\ell+1)}\mathbf{z}^{(\ell+1)}, \sigma_\ell^2 \mathbf{I}), \quad (25)$$

а найнижчий рівень генерує передбачення для байтового субстрату:

$$p(s|\mathbf{z}^{(1)}) = \prod_{i=1}^N \text{Cat}(b_i; \text{softmax}(\mathbf{V}\mathbf{z}_i^{(1)})), \quad (26)$$

де $\text{Cat}(\cdot; \mathbf{p})$ — категоріальний розподіл з параметрами $\mathbf{p} \in \Delta^{255}$ (симплекс розмірності 255), а $\mathbf{V} \in \mathbb{R}^{256 \times d_1}$ — матриця генерації.

Вільна енергія для цієї моделі обчислюється як:

$$\mathcal{F}_{\text{free}} = \underbrace{D_{\text{KL}}(q(\mathbf{z})||p(\mathbf{z}|s))}_{\text{складність моделі}} - \underbrace{\mathbb{E}_{q(\mathbf{z})}[\log p(s|\mathbf{z})]}_{\text{точність передбачення}}, \quad (27)$$

де $q(\mathbf{z})$ — варіаційний апроксиматор апостеріорного розподілу, який параметризується станом польової системи. Мінімізація вільної енергії одночасно зменшує похибку передбачення (другий доданок) та обмежує складність моделі (перший доданок).

6.2 Інформаційне вузьке місце як дизайн-принцип

Принцип інформаційного вузького місця (Information Bottleneck, IB) забезпечує теоретичну основу для формування оптимальних абстракцій у БКС. Для байтового потоку S (вхід) та цільової змінної Y (наприклад, наступні байти або мета завдання) ми шукаємо стиснене представлення T , яке максимізує взаємну інформацію з Y при обмеженні на взаємну інформацію з S :

$$\min_{p(T|S)} I(S; T) - \beta I(T; Y), \quad (28)$$

де $\beta > 0$ — параметр Лагранжа, що контролює компроміс між стисненням та інформативністю, $I(\cdot; \cdot)$ — взаємна інформація Шеннона.

У контексті БКС цей принцип діє на кожному рівні конвертації. Когнітивний кластер рівня ℓ є стисненим представленням кластерів рівня $\ell - 1$, яке зберігає максимальну інформацію про цільову змінну (контекст завдання або майбутні байти). При $\beta \rightarrow 0$ представлення стає максимально стисненим (втрачає інформативність), а при $\beta \rightarrow \infty$ — максимально інформативним (але не стисненим). Оптимальне значення β відповідає фазовому переходу в інформаційному просторі, аналогічно до того, як температура T_c відповідає фазовому переходу в просторі польової динаміки.

6.3 Емерджентна оптимізація

На відміну від традиційних нейронних мереж, де оптимізація здійснюється єдиним алгоритмом (зазвичай варіантом градієнтного спуску), БКС використовує багаторівневу емерджентну оптимізацію, де кожен рівень має власну динаміку навчання.

На рівні байтового субстрату оптимізація відбувається через польову динаміку — зміна параметрів поля (D_k, θ_k, μ) відповідно до градієнта вільної енергії:

$$\frac{d\theta_k}{dt} = -\eta_\theta \frac{\partial \mathcal{F}_{\text{free}}}{\partial \theta_k}, \quad \frac{dD_k}{dt} = -\eta_D \frac{\partial \mathcal{F}_{\text{free}}}{\partial D_k}. \quad (29)$$

На рівні тензорів взаємодії оптимізація відбувається через градієнтний спуск за параметрами $\mathbf{W}_\beta, \mathbf{A}$ та векторами байтових значень $\mathbf{u}_k$:

$$\mathbf{W}_\beta \leftarrow \mathbf{W}_\beta - \eta_W \nabla_{\mathbf{W}_\beta} \mathcal{F}_{\text{free}}, \quad \mathbf{u}_k \leftarrow \mathbf{u}_k - \eta_u \nabla_{\mathbf{u}_k} \mathcal{F}_{\text{free}}. \quad (30)$$

На рівні конвертаційних шарів оптимізація відбувається через варіаційний висновок (variational inference), що мінімізує розрив між апроксиматором q та істинним апостеріорним розподілом. Нарешті, на найвищому рівні — рівні “мета-навчання” — система оптимізує власну архітектуру: кількість конвертаційних шарів, розмірність представлень d_ℓ , та параметри фазових переходів $T_{c,\ell}$.

Така ієрархічна оптимізація має важливу перевагу: кожен рівень працює з власним масштабом часу. Субстрат адаптується швидко (мілісекунди), тензори — помірно (секунди), конвертаційні шари — повільно (хвилини), а мета-навчання — дуже повільно (години). Це відповідає біологічній реальності, де синаптична пластичність (мілісекунди-секунди) та структурна реорганізація (години-дні) працюють на різних часових масштабах.

7 Порівняльний аналіз з існуючими підходами

У цьому розділі ми зіставляємо архітектуру БКС з основними існуючими парадигмами III за низкою критеріїв. Результати порівняння узагальнено у таблиці 1.

Таблиця 1: Порівняльний аналіз архітектур III.

Критерій	Трансформер	CNN	RNN	Нейроеволюція	БКС
Одиниця обробки	Токен	Піксель/патч	Токен	Нейрон	Байт
Попередня обробка	Токенізація	Нормалізація	Токенізація	Мінімальна	Відсутня
Модальність	Залежна	Залежна	Залежна	Незалежна	Незалежна
Механізм уваги	Фіксований	Відсутній	Неявний	Емерджентний	Польовий
Абстракції	Задані архітектурою	Задані архітектурою	Задані архітектурою	Емерджентні	Самоорганізовані
Масштабованість	Квадратична	Лінійна	Лінійна	Змінна	Суб-квадратична

7.1 Трансформери

Трансформери є домінуючою архітектурою сучасного III, але вони мають ряд фундаментальних обмежень, які БКС покликаний подолати. По-перше, механізм уваги трансформера працює з фіксованим словником токенів, який формується апіорі через токенізатор (наприклад, BPE, WordPiece). Це означає, що система не може виявити структуру, яка не відповідає жодному токенові словника. По-друге, складність механізму уваги є квадратичною відносно довжини послідовності $O(N^2)$, що обмежує довжину контексту. У БКС польова взаємодія має складність $O(N \cdot K)$, де K — середня кількість “активних” взаємодій на байт, причому $K \ll N$ завдяки адаптивній довжині взаємодії λ . По-третє, трансформери не мають власної динаміки — вони є “статичними” функціями від входу до виходу, без внутрішнього стану, який еволюціонує в часі. БКС, навпаки, має багату внутрішню динаміку через польові рівняння.

7.2 Згорткові нейронні мережі

Згорткові нейронні мережі (CNN) є ближчими до БКС за одним аспектом: вони також працюють з “локальними” взаємодіями (через згорткові фільтри). Однак CNN мають жорстко фіксований рецептивний розмір (kernel size), тоді як у БКС рецептивне поле є адаптивним і визначається параметром $\lambda(\mathbf{h}_i, \mathbf{h}_j)$. Крім того, CNN спеціалізовані на даних з регулярною ґратковою структурою (зображення, аудіо), тоді як БКС працює з довільними байтовими послідовностями, не припускаючи жодної просторової структури.

7.3 Рекурентні архітектури

Рекурентні нейронні мережі (RNN), включаючи LSTM та GRU, обробляють послідовності послідовно, підтримуючи прихований стан. У цьому відношенні во-

ни є ближчими до байтового підходу, оскільки теоретично можуть обробляти байтові послідовності. Однак RNN мають дві критичні проблеми: (1) проблему зникаючого/вибухового градієнта, яка практично обмежує довжину контексту, і (2) жорстко послідовну обробку, яка унеможлиблює паралелізацію. БКС вирішує обидві проблеми: польова взаємодія є локальною (не вимагає проходження через весь ланцюг), а оновлення поля може виконуватися паралельно для всіх позицій.

7.4 Нейроеволюційні підходи

Нейроеволюційні методи (NEAT, HyperNEAT та ін.) є найближчими до БКС за духом, оскільки вони також прагнуть емерджентної архітектури. Однак нейроеволюція оперує на рівні окремих нейронів та зв'язків, що робить пошуковий простір надзвичайно великим і практично не масштабується за межами простих завдань. БКС працює на рівні байтів та кластерів, що є більш абстрактним і дозволяє ефективніше досліджувати простір архітектур.

8 Шляхи реалізації

8.1 Обчислювальні вимоги

Архітектура БКС висуває специфічні обчислювальні вимоги, які відрізняються від вимог традиційних нейронних мереж. По-перше, польова динаміка вимагає ітеративного обчислення: на кожному кроці часу необхідно оновити поле активації для всіх позицій та байтових значень. По-перше, для байтового потоку довжиною N кожна ітерація вимагає $O(N \cdot 256)$ операцій. При $N = 10^6$ (1 МБ даних) та 100 ітераціях це становить приблизно 2.56×10^{10} операцій, що цілком реально на сучасних GPU.

По-друге, обчислення тензорів взаємодії вимагає $O(N \cdot K)$ операцій, де K — середня кількість активних взаємодій. Завдяки адаптивній довжині взаємодії λ , K може бути значно меншим за N : для структурованих даних типові значення K складають 10^2 – 10^3 , тоді як N може бути 10^6 – 10^9 . Це робить обчислення субквадратичним, на відміну від трансформерів з $O(N^2)$.

8.2 Апаратна архітектура

Ефективна реалізація БКС вимагає апаратної архітектури, оптимізованої для двох типів операцій: (1) масового паралельного оновлення поля активації, та (2) розріджених матричних операцій для тензорів взаємодії. Для першого типу ідеально підходять GPU або TPU з великою кількістю ядер та високою пропускною здатністю пам'яті. Для другого типу перспективними є процесори з розрідженими матричними блоками (sparse matrix units), які стають все більш поширеними у сучасних прискорювачах.

Окремої уваги вимагає питання пам'яті. Байтовий потік довжиною N вимагає зберігання поля активації розміром $N \times 256$ дійсних чисел. При $N = 10^6$ та 4 байтах на число це становить приблизно 1 ГБ. Для більших потоків ($N = 10^9$) це зростає до 1 ТБ, що вимагає механізмів сегментації та підкачки поля. Ми пропонуємо використовувати вікнову обробку з перетинанням: поле активації

обчислюється лише для поточного вікна розміром W , причому перетин між сусідніми вікнами забезпечує неперервність контексту.

8.3 Етапи розробки

Ми пропонуємо такі етапи розробки БКС.

Етап 1: Демонстратор байтового субстрату. Реалізація базового байтового поля з фіксованою функцією взаємодії на невеликих даних (до 1 КБ). Мета — перевірити, чи виникають когнітивні кластери в простих байтових потоках (ASCII-текст, прості зображення BMP).

Етап 2: Адаптивні тензори взаємодії. Реалізація параметрів $\mathbf{W}_\beta$, $\mathbf{A}$, $\mathbf{u}_k$, що навчаються, та перевірка здатності системи виявляти структуру у даних різних модальностей.

Етап 3: Конвертаційні шари. Реалізація ієрархічної конвертації та перевірка формування вищорівневих абстракцій.

Етап 4: Масштабування. Перехід до байтових потоків розміром до 1 МБ, оптимізація продуктивності, реалізація вікнової обробки.

Етап 5: Багатомодальне навчання. Перевірка модальність-незалежної обробки на суміші текстових, зображувальних та аудіоданих.

Етап 6: Порівняльні експерименти. Зіставлення з трансформерами та іншими архітектурами на стандартних бенчмарках.

Таблиця 2: Етапи розробки БКС та оціночні терміни.

Етап	Назва	Ключові завдання	Термін
1	Субстрат	Байтове поле, фіксована взаємодія	3–4 міс.
2	Тензори	Параметризація, навчання	4–6 міс.
3	Конвертація	Ієрархія, зворотний зв'язок	6–8 міс.
4	Масштабування	Віконна обробка, GPU	4–6 міс.
5	Мультимодальність	Змішані дані	4–6 міс.
6	Бенчмарк	Порівняння з SOTA	3–4 міс.

9 Накопичення досвіду та пам'ять

Польове рівняння (9) є транзієнтним: байти пройшли через систему, поле розсіялося, і сліду не залишилося. Без механізму накопичення досвіду БКС була б системою без пам'яті, яка обробляє кожен новий вхід як уперше. У цьому розділі ми розробляємо архітектуру пам'яті БКС, засновану на двох типах зберігання, аналогічних біологічним системам.

9.1 Кристалізована пам'ять (довгострокова)

Когнітивний кластер, який зустрічається повторно, укріплюється і переходить у стабільний стан. Ми визначаємо кристалізований банк $\mathcal{K} = \{K_1, K_2, \dots\}$, де кожен кристал $K_j = (\mathbf{p}_j, \mathbf{h}_j, n_j, \tau_j)$ містить: розподіл байтових значень $\mathbf{p}_j$, векторне

представлення вищого рівня $\mathbf{h}_j$, кількість зустрічей n_j та час останньої зустрічі τ_j .

Визначення 9.1 (Правило консолідації). Кластер C консолідується у кристал K_j , якщо його активація перевищує поріг Θ_{cryst} протягом $\Delta t > \tau_{\text{cryst}}$. Правило оновлення:

$$n_j(t+1) = \begin{cases} n_j(t) + 1, & \text{якщо } K_j \text{ впізнаний,} \\ n_j(t) \cdot e^{-\Delta t / \tau_{\text{decay}}}, & \text{інакше.} \end{cases} \quad (31)$$

Кристал з $n_j < n_{\min}$ розчиняється (забувається).

Це правило моделює довгострокове потенціювання (LTP) у біологічних синапсах: повторна активація укріплює зв'язок, тоді як відсутність активації веде до поступового занепаду. Параметр τ_{decay} визначає швидкість забування і має бути суттєво більшим за τ_{cryst} , щоб уникнути передчасного розчинення кристалів.

9.2 Робоча пам'ять (короткострокова)

Для поточного контексту ми визначаємо кільцевий буфер кластерів обмеженого розміру M :

$$\mathcal{W}(t) = [C_{t-M+1}, C_{t-M+2}, \dots, C_t]. \quad (32)$$

Кластери в робочій пам'яті мають вагу релевантності, яка залежить від трьох факторів: часу (свіжіший кластер важливіший), зв'язку з поточним вхідним потоком (через функцію контекстної спорідненості γ), та значущості (через глибину атракторного басейну, в який поле потрапило при обробці даного кластера). Формально вага кластера C_i у робочій пам'яті:

$$w_i(t) = e^{-(t-t_i)/\tau_{\text{WM}}} \cdot \gamma(\mathbf{h}_i, \mathbf{h}_{\text{current}}) \cdot \omega_{\text{attract}}(C_i), \quad (33)$$

де t_i — час надходження кластера, τ_{WM} — стала часу робочої пам'яті, а ω_{attract} — глибина атракторного басейну.

9.3 Впізнавання через атракторні басейни

Ключове питання: як саме система впізнає раніше бачений кластер? Пряме порівняння KL-дивергенції між новим кластером та усіма кристалами банку є обчислювально нездійсненним при великому розмірі банку, а також страждає від асиметрії KL-дивергенції та проблеми нульових ймовірностей.

Ми пропонуємо принципово інший підхід: впізнавання є властивістю польової динаміки, а не окремою операцією пошуку. Кожен кристалізований кластер K_j створює в полі атракторний басейн — область простору станів, яка “притягує” поле, якщо вхід схожий на цей кристал. Кристал модифікує функцію реакції через додатковий потенціал:

$$V_{\text{attract}}(i, k, t) = - \sum_{j \in \mathcal{K}} \omega_j \cdot \exp\left(-\frac{\|\mathbf{h}_i^{(t)} - \mathbf{h}_j\|^2}{2\sigma_j^2}\right) \cdot \delta(k - k_j^*), \quad (34)$$

де $\omega_j = \alpha(n_j)$ — глибина потенціальної ями (пропорційна частоті зустрічей кристалів), σ_j — ширина ями (обернено пропорційна n_j : чим частіше бачили, тим точніше впізнавання), k_j^* — пікове байтове значення кристала.

Коли нові байти надходять, поле еволюціонує згідно з (9), доповненим атракторним потенціалом (34). Результат впізнавання визначається тим, куди релаксує поле:

Таблиця 3: Результати впізнавання через атракторну релаксацію поля.

Стан після релаксації	Інтерпретація
Поле впало в басейн K_j	Впізнано $n_j + = 1$, підсилити ω_j
Поле на межі $K_j, K_{j'}$	Амбівалентний гібридний кристал
Поле не впало в жоден басейн	Нове Створити новий кристал K_{new}

Амбівалентний випадок є особливо важливим: система створює гібридний кристал, який успадковує властивості обох батьківських кристалів. Це основа метафоричного мислення — здатності бачити один концепт крізь призму іншого.

10 Контекстна архітектура

БКС обробляє байти локально, через механізм польової взаємодії з адаптивною довжиною λ . Однак локальності недостатньо для розуміння послідовностей, утримання теми розмови або виявлення трендів. У цьому розділі ми розробляємо тривірневу контекстну архітектуру.

10.1 Локальний контекст: польова взаємодія

Локальний контекст забезпечується механізмом, який вже описано: польова взаємодія з адаптивним $\lambda(\mathbf{h}_i, \mathbf{h}_j)$ дає контекст у межах вікна. Це аналог периферичного зору — система “бачить” навколо поточної позиції, але не далі. Для структурованих ділянок даних (наприклад, окремі слова в тексті) λ мала і контекст локальний; для монотонних або періодичних ділянок λ зростає і контекст розширюється.

10.2 Послідовний контекст: контекстний резонанс

Система має знати: “я зараз у середині розмови про X, до цього було Y, потім Z”. Для цього ми вводимо механізм контекстного резонансу. Кожен новий кластер C_t резонує з кристалізованим банком:

$$r_j(t) = \gamma(\mathbf{h}_t, \mathbf{h}_j) \cdot \alpha(n_j) \cdot e^{-\Delta t_j / \tau_{\text{context}}}, \quad (35)$$

де j — індекс кристала в банку, $\alpha(n_j)$ — функція підсилення (чим частіше бачили, тим сильніший резонанс), а τ_{context} — часова стала контексту.

Контекстний вектор обчислюється як зважена сума резонуючих кристалів:

$$\mathbf{ctx}(t) = \sum_{j \in \mathcal{K}} r_j(t) \cdot \mathbf{h}_j. \quad (36)$$

Цей вектор підмішується до польового рівняння як додатковий член — контекстна ін'єкція:

$$\frac{\partial \Phi(i, k, t)}{\partial t} = [\text{дифузія}] + [\text{реакція}] - [\text{затухання}] + \underbrace{\kappa \cdot \text{ctx}(t) \cdot \sigma_{\text{gate}}(\text{ctx}, \Phi)}_{\text{контекстна ін'єкція}}, \quad (37)$$

де κ — коефіцієнт сили контекстної ін'єкції, а σ_{gate} — функція ворот (gating), яка запобігає неконтрольованому впливу контексту. Суть: контекст модифікує польову динаміку. Якщо ми в контексті “розмова про котів”, поле стає більш чутливим до байтів, що кодують котячу тематику. Це не attention у сенсі трансформерів — це зміна самої динаміки обробки.

10.3 Накопичувальний контекст: гладкий інерційний стан

Для розуміння трендів на масштабі всього документа (наприклад, “протягом останніх 500 слів автор дедалі більше розчаровується”) ми вводимо гладкий контекстний стан $\mathbf{s}(t)$, який еволюціонує повільніше за поле:

$$\mathbf{s}(t+1) = \beta_s \cdot \mathbf{s}(t) + (1 - \beta_s) \cdot \text{ctx}(t), \quad (38)$$

де $\beta_s \approx 0.99$ — коефіцієнт інерції. Цей стан інтегрує контекст з великим часовим вікном, згладжуючи локальні флуктуації. Він відповідає за розуміння трендів, тональності, теми на масштабі всього документа.

11 Часові масштаби та їх узгодження

Кожен механізм БКС працює на своєму часовому масштабі, і ці масштаби мають бути узгоджені для забезпечення стабільності системи.

11.1 Елементарний такт та шкала часу

Визначимо елементарний такт τ_0 — час одного оновлення польового рівняння. На GPU це порядку $\tau_0 \approx 0.1$ –1 мс. Усі інші часові сталі виражаються через τ_0 .

11.2 Умова розділення масштабів

Кожна наступна часова стала приблизно на порядок–два більша за попередню. Це не випадковість, а умова стабільності. Якщо контекстний стан змінюється так само швидко, як поле, немає контексту — є лише шум. Якщо кристалізація відбувається за один такт, система запам'ятовує все, включно з шумом. Формально:

$$\tau_{k+1} \geq 10 \cdot \tau_k, \quad (39)$$

для кожної пари сусідніх механізмів. Це аналог критерію Куранта у чисельних методах: часовий крок має бути менший за час поширення сигналу через один просторовий вузол, інакше виникає обчислювальна нестійкість.

Таблиця 4: Часові масштаби механізмів БКС.

Механізм	Опис	У тактах τ_0	Аналог у мозку
Полюова релаксація	Збіжність поля до атрактора	10–50	Нейронна інтеграція
Формування кластера	Виникнення стабільного кластера	50–200	Зв’язкова активація
Робоча пам’ять	Утримання поточного контексту	10^3 – 10^4	Робоча пам’ять
Контекстний стан	Послідовний контекст	10^4 – 10^5	Утримання контексту
Кристалізація	Перехід у довгострокову пам’ять	10^5 – 10^6	Консолідація пам’яті
Розщеплення рівнів	Структурна реорганізація	10^6 – 10^8	Структурна пластичність
Забування	Розчинення кристалів	10^7 – 10^9	Довгострокове забування

11.3 Адаптивний елементарний такт

Різні модальності даних надходять з різною швидкістю: текст — послідовно байт за байтом, зображення — мегабайти одночасно, аудіо — потоком з фіксованою частотою. Щоб τ_0 був осмисленим для всіх модальностей, ми прив’язуємо його до щільності входу:

$$\tau_0(t) = \frac{\tau_{\text{base}}}{\max(1, |\Delta B(t)|)}, \quad (40)$$

де $|\Delta B(t)|$ — кількість нових байтів за останній фізичний інтервал. Для тексту: $|\Delta B| = 1 \Rightarrow \tau_0 = \tau_{\text{base}}$. Для зображення: $|\Delta B| = 10^6 \Rightarrow \tau_0 = 10^{-6} \cdot \tau_{\text{base}}$ (поле оновлюється швидше, щоб “перетравити” масив даних).

12 Перенесення знань між модальностями

Як знання про “кіт” у текстовому контексті переноситься на зображення кота? Пряма спільна зустріч — занадто сильна умова, яка рідко виконується на практиці. У цьому розділі ми розробляємо чотири механізми перенесення, кожен з яких працює за різних умов.

12.1 Механізм 1: Структурний ізоморфізм

Два концепти з різних модальностей можуть мати однакову внутрішню структуру навіть без спільної зустрічі. Наприклад, слово “кіт” у тексті та зображення кота асоціюються з наборами ознак, що перетинаються: {м’який, вуса,

хвіст, маленьке}. На високому рівні конвертації ці ознаки представлені векторами, і перенесення відбувається через їхню косинусну подібність:

$$\cos(\mathbf{f}_{m_1}^{(\ell)}, \mathbf{f}_{m_2}^{(\ell)}) > \theta_{\text{transfer}}, \quad (41)$$

де $\mathbf{f}_m^{(\ell)}$ — вектор ознак кластера з модальності m на рівні ℓ . Цей механізм працює без жодної спільної зустрічі — лише через подібність у просторі ознак.

12.2 Механізм 2: Універсальний байтовий скаффолд

Простори ознак різних модальностей стають сумісними завдяки тому, що обидві модальності обробляються однією і тією ж архітектурою: однакові тензорні механізми ($\mathbf{W}_\beta$, $\mathbf{A}$), однакові правила польової динаміки (D_k , μ), однакові правила кристалізації та конвертації. Ця однорідність створює структурний скаффолд — універсальні принципи, вбудовані в архітектуру:

- **Суміжність** → **групування**: сусідні байти групуються (сусідні символи → слова, сусідні пікселі → текстури).
- **Повторення** → **патерн**: повторювані структури впізнаються (повторювані слова, повторювані текстури).
- **Межа** → **сегментація**: різкі переходи розділяють (пробіли, краї об'єктів).
- **Контрастність** → **виділення**: аномалії привертають увагу.

Ці принципи не навчаються — вони є наслідком архітектури, і вони гарантують, що простори ознак різних модальностей будуть сумісними.

12.3 Механізм 3: Ланцюгове перенесення

Навіть якщо модальності m_1 та m_3 ніколи не зустрічаються, перенесення можливе через проміжну модальність m_2 :

$$m_1 \xrightarrow{w_{1,2}} m_2 \xrightarrow{w_{2,3}} m_3 \implies w_{1,3}^{\text{indirect}} = w_{1,2} \cdot w_{2,3}. \quad (42)$$

Приклад: “кіт” (текст) співіснує зі звуком нявкання (аудіо), звук нявкання співіснує з зображенням кота (візуал). Тепер “кіт” та зображення пов’язані через аудіо-міст, хоча ніколи не зустрічалися разом. Це транзитивність перенесення, яка дозволяє системі поступово будувати мережу концептів.

12.4 Механізм 4: Мовне заземлення

Мова є унікальною модальністю, оскільки вона описує інші модальності: “червоний” зв’язується з візуальним червоним, “гучний” — з аудіальним гучним, “м’який” — з тактильним. Мова працює як мета-модальність, яка забезпечує міжмодальне вирівнювання. Навіть якщо два зображення ніколи не зустрічалися разом, вони можуть бути пов’язані через спільний текстовий опис. Формально, мовне заземлення діє через зворотний зв’язок: коли вищі конвертаційні шари ідентифікують мовний кластер, що описує немовний концепт, вони створюють прямий зв’язок між відповідними кристалами в банку.

13 Зростання складності абстракцій: автокаталітичне розщеплення

БКС не має фіксованої кількості конвертаційних шарів. Шари народжуються, коли система цього потребує, і зникають, коли стають надлишковими. У цьому розділі ми детально описуємо механізм розщеплення рівнів, крок за кроком.

13.1 Крок 0: Детекція потреби

Моніторимо статистику міжкластерних відстаней на рівні ℓ :

$$\mathcal{D}_\ell = \{D_{\text{KL}}(\mathbf{p}_i^{(\ell)} \parallel \mathbf{p}_j^{(\ell)}) : i < j, i, j \in \mathcal{K}_\ell\}. \quad (43)$$

Якщо розподіл $\mathcal{D}_\ell$ стає бімодальним — є група дуже схожих кластерів і група дуже різних — це сигнал до розщеплення. Для кількісної оцінки бімодальності використовуємо коефіцієнт Сарлса:

$$b = \frac{g^2 + 1}{\kappa}, \quad (44)$$

де g — асиметрія, κ — ексцес розподілу $\mathcal{D}_\ell$. Якщо $b > 0.555$ — розподіл бімодальний, і запускається процедура розщеплення.

13.2 Крок 1: Побудова графа подібності

Створюємо граф $G_\ell = (V, E, W)$, де $V = \mathcal{K}_\ell$ (вузли — кластери рівня ℓ), E — ребра між усіма парами, а ваги:

$$W_{ij} = \exp(-D_{\text{KL}}(\mathbf{p}_i \parallel \mathbf{p}_j)). \quad (45)$$

13.3 Крок 2: Спектральна кластеризація

Обчислюємо нормалізований лапласіан графа:

$$L_{\text{norm}} = I - D^{-1/2} W D^{-1/2}, \quad (46)$$

де D — діагональна матриця ступенів. Знаходимо власні значення $\lambda_1 \leq \lambda_2 \leq \dots$ та власні вектори $\mathbf{v}_1, \mathbf{v}_2, \dots$. Проміжок між власними значеннями $\lambda_{k+1} - \lambda_k$ визначає кількість нових груп. Проектуємо вузли у простір власних векторів:

$$\mathbf{y}_i = (v_1(i), v_2(i), \dots, v_k(i)) \in \mathbb{R}^k, \quad (47)$$

і групуємо k-means. Результат: розбиття $\mathcal{K}_\ell = G_1 \sqcup G_2 \sqcup \dots \sqcup G_k$.

13.4 Крок 3: Створення нового рівня

Створюємо проміжний рівень ℓ' між ℓ та $\ell + 1$. Кожна група G_m стає одним кластером нового рівня: $C_m^{(\ell')} = \text{Agg}(\{C_i^{(\ell)} : i \in G_m\})$. Параметри нового рівня ініціалізуються: представлення $\mathbf{h}_m^{(\ell')} = \frac{1}{|G_m|} \sum_{i \in G_m} \mathbf{h}_i^{(\ell)}$ (центроїд групи), матриця перетворення $\mathbf{W}_{\ell'}^{(1)} = \mathbf{I}$ (тотожне перетворення), зв'язки вниз (кожен $C_i^{(\ell)}$ пов'язаний лише з $C_m^{(\ell')}$, де $i \in G_m$), зв'язки вгору ($C_m^{(\ell')}$ успадковує зв'язки від своїх дітей до рівня $\ell + 1$).

13.5 Крок 4: Калібрація

Новий рівень ℓ' проходить період високої пластичності (аналог критичного періоду в нейробіології):

$$\eta_{\ell'} = \eta_{\text{base}} \cdot (1 + \alpha_{\text{calib}} \cdot e^{-t/\tau_{\text{calib}}}), \quad (48)$$

де $\alpha_{\text{calib}} \approx 10$ — множник швидкості навчання під час калібрації, τ_{calib} — час калібрації. Під час калібрації градієнти вільної енергії тече через новий рівень, уточнюючи $\mathbf{W}_{\ell'}^{(1)}$, кластери можуть мігрувати між групами, а параметри сусідніх рівнів також підлаштовуються.

13.6 Крок 5: Валідація через $\Delta\mathcal{F}_{\text{free}}$

Після калібрації перевіряємо, чи дійсно новий рівень зменшив вільну енергію:

$$\Delta\mathcal{F}_{\text{free}} = \mathcal{F}_{\text{free}}^{\text{after}} - \mathcal{F}_{\text{free}}^{\text{before}}. \quad (49)$$

Якщо $\Delta\mathcal{F}_{\text{free}} < -\epsilon$ — розщеплення успішне, рівень залишається. Якщо $\Delta\mathcal{F}_{\text{free}} \geq -\epsilon$ — розщеплення не дало виграшу, рівень видаляється (зрощення назад), система повертається до попереднього стану. Це аналог біологічного процесу: не кожна мутація корисна, система “пробує” розщеплення і залишає його лише за емпіричного підтвердження.

13.7 Зрощення надмірних рівнів

Зворотний процес: якщо два кластери на сусідніх рівнях завжди активуються разом, $\mathbb{P}(C_i^{(\ell)} | C_j^{(\ell+1)}) > 1 - \epsilon$, вони зростаються: проміжний рівень зникає, і система стає компактнішою. Це аналог pruning в нейроеволюції, але органічний — система самостійно виявляє надмірність і усуває її.

14 Потенційні проблеми та шляхи їх вирішення

Архітектура БКС є радикальним відходом від усталених підходів, і такий відхід неминуче породжує нові проблеми. У цьому розділі ми розглядаємо сім ключових проблем, які можуть виникнути при реалізації БКС, та пропонуємо конкретні шляхи їх вирішення.

14.1 Проблема масштабованості байтового поля

Польове рівняння (9) вимагає зберігання поля активації розміром $N \times 256$ для потоку довжиною N . При $N = 10^6$ (1 МБ) це 1 ГБ пам'яті; при $N = 10^9$ (1 ГБ) — 1 ТБ. Пряма реалізація неможлива для великих потоків.

Рішення: **ієрархічна віконна обробка з перетинанням**. Поле обчислюється лише для поточного вікна розміром W , причому перетин між сусідніми вікнами $\delta W \geq \lambda_{\text{max}}$ забезпечує неперервність контексту. Кристали, що сформувалися у попередньому вікні, зберігаються в банку і впливають на нове вікно

через механізм контекстного резонансу (35). Додатково, для дуже великих потоків можна використовувати розріджене поле: зберігати лише позиції з активацією вище порогу $\phi_{\min}$, оскільки більшість позицій матиме близьку до нуля активацію завдяки затуханню μ .

14.2 Проблема вибору початкових параметрів

Польова динаміка чутлива до параметрів D_k , θ_k , μ , a_k . Невідповідність може призвести до відсутності самоорганізації (занадто велике μ або занадто малі D_k) або до хаотичної поведінки (занадто мале μ або занадто великі D_k).

Рішення: **мета-оптимізація на рівні популяцій**. Замість ручного підбору параметрів, ми використовуємо еволюційну стратегію (СМА-ES), яка шукає оптимальні параметри в просторі $\Theta = \{D_k, \theta_k, \mu, a_k\}$. Фітнес-функція — швидкість зменшення вільної енергії на валідаційному наборі байтових потоків. Початковий діапазон пошуку обмежується умовою стабільності $\omega(q, k) < 0$ для всіх q (відсутність нестійкості при $T > T_c$).

14.3 Проблема “дитячої амнезії”

На ранніх стадіях навчання кристалізований банк порожній. Система не має атракторних басейнів, поле не впізнає жодних патернів, і кожен новий вхід створює новий кристал. Банк швидко переповнюється шумовими кристалами, які згодом розчиняються, але витрачають обчислювальні ресурси.

Рішення: **стадія імпринтингу**. На початку навчання система працює в спеціальному режимі: високий поріг кристалізації Θ_{cryst} (кристалізуються лише найсильніші кластери), швидке забування τ_{decay} (шумові кристали швидко розчиняються), та підвищена увага до повторень (кластер, що з’явився ≥ 3 разів, кристалізується негайно, оминаючи стандартний поріг). Це аналог критичного періоду розвитку в нейробіології: мозок дитини має підвищену пластичність, але також і підвищену “вибірковість” до стимулів.

14.4 Проблема катастрофічного забування

Коли система зустрічає новий тип даних, нові кристали можуть витіснити старі, особливо якщо банк обмежений за розміром. Це аналог катастрофічного забування в нейронних мережах.

Рішення: **еластична консолідація**. Для кожного кристала K_j з високим n_j (часто зустрічаний) ми збільшуємо його “міцність”: глибина атракторної ями ω_j пропорційна $\log(n_j + 1)$, а ширина σ_j обернено пропорційна $\sqrt{n_j}$. Це означає, що старі добре укріплені кристали важче “перемістити” новими даними. Додатково, при досягненні банком максимального розміру $|\mathcal{K}| = K_{\max}$, новий кристал заміщує не найстаріший, а той, що має найменше $n_j \cdot \omega_j$ (найменше “інвестовано”).

14.5 Проблема шуму та нерелевантних даних

Не всі байти несуть корисну інформацію. Заголовки файлів, метадані, випадковий шум — все це породжує кластери, які не відповідають змістовним патер-

нам. Якщо система кристалізує кожен такий кластер, банк забивається сміттям.

Рішення: **двохетапна фільтрація**. Перший етап — поріг активації: кластер C розглядається як кандидат на кристалізацію лише якщо його пікова активація $\max_k \phi(i, k, t) > \Theta_{\text{active}}$. Другий етап — перевірка стійкості: кластер має витримати ≥ 3 циклів польової релаксації без суттєвої зміни форми (KL-дивергенція між послідовними станами менша за ϵ_{stable}). Шумові кластери, як правило, нестійкі — вони “розпливаються” при повторній обробці.

14.6 Проблема упередженості кодування

Різні кодування даних створюють різні байтові представлення одного й того ж змісту (наприклад, UTF-8 vs UTF-16 для тексту, PNG vs JPEG для зображень). Якщо система навчилася на UTF-8 тексті, вона може не впізнати той самий текст у UTF-16.

Рішення: **виявлення кодування через сигнатури**. Байтовий потік починається з “магічних чисел” (BOM для UTF, сигнатури PNG/JPEG тощо), які система може виявити як окремі кластери-маркери. Після виявлення маркера кодування система адаптує параметри поля: наприклад, для UTF-16 довжина взаємодії λ подвоюється (кожен символ — 2 байти замість 1), а функція сумісності β налаштовується на парні байтові послідовності. Ця адаптація відбувається автоматично через механізм модальної ідентифікації (24).

14.7 Проблема обчислювальної ефективності атракторних байсейнів

Формула (34) вимагає сумування по всіх кристалах банку $\mathcal{K}$ для кожної позиції i та кожного байтового значення k . При $|\mathcal{K}| = 10^6$ та $N = 10^5$ це 10^{11} операцій на кожен такт.

Рішення: **приблизна індексація через локально-чутливе хешування (LSH)**. Замість повного сканування банку, ми хешуємо вектор $\mathbf{h}_i^{(t)}$ та отримуємо короткий список кандидатів $\mathcal{K}_{\text{cand}} \subset \mathcal{K}$ з $|\mathcal{K}_{\text{cand}}| \ll |\mathcal{K}|$. Формула (34) обчислюється лише по $\mathcal{K}_{\text{cand}}$. LSH гарантовано знаходить усі кристали з $\|\mathbf{h}_i - \mathbf{h}_j\| < r$ з ймовірністю $\geq 1 - \delta_{\text{LSH}}$, де r та δ_{LSH} — параметри хешування. Типове значення: $|\mathcal{K}_{\text{cand}}| \approx 100$ замість 10^6 , що зменшує обчислювальну складність на 4 порядки.

15 Висновки та перспективи

У даній роботі ми представили архітектуру Байт-Когнітивного Субстрату (БКС) — принципово новий підхід до побудови систем штучного інтелекту, заснований на трьох ключових ідеях: (1) байт як фундаментальна одиниця обробки, (2) польова взаємодія як механізм формування зв'язків, та (3) самоорганізація як джерело абстракцій.

Розроблений математичний формалізм включає тензорний опис байтової взаємодії, польові рівняння реакції-дифузії, адаптовані для байтового рівня, інформаційну геометрію простору байтових розподілів, та динаміку самооргані-

зації на основі принципу мінімізації вільної енергії. Цей формалізм забезпечує теоретичну основу для аналізу та реалізації БКС.

Архітектура БКС має ряд потенційних переваг перед існуючими підходами. По-перше, вона є модальність-незалежною: одна й та сама архітектура обробляє текст, зображення, аудіо та будь-які інші цифрові дані без модифікації. По-друге, вона не потребує токенизації чи іншої попередньої обробки, що усуває фундаментальне джерело втрати інформації. По-третє, абстракції формуються самою системою через механізми самоорганізації, що відкриває можливість для виявлення структур, які не передбачені розробником.

Серед відкритих питань та напрямків подальших досліджень варто виділити такі. По-перше, ефективність польової динаміки вимагає експериментальної перевірки: чи дійсно когнітивні кластери, що виникають через фазові переходи, відповідають змістовним структурам даних? По-друге, масштабованість архітектури потребує додаткового аналізу: як поведінка системи змінюється при переході від кілобайтів до гігабайтів? По-третє, питання навчання залишається відкритим: які алгоритми оптимізації є найбільш ефективними для параметрів тензорів взаємодії та конвертаційних шарів?

Ми вважаємо, що архітектура БКС відкриває новий напрямок досліджень у галузі штучного інтелекту — напрямок, заснований не на вдосконаленні існуючих парадигм, а на фундаментально іншому підході до представлення та обробки інформації. Принцип “дай системі реальну інформацію і дозволь їй самій знайти в ній структуру” може виявитися ключовим для створення систем, здатних до справжнього інтелекту.